

```
C:\cygwin64\home\deepu
The current date is: 02-10-2019
Enter the new date: (dd-mm-yy)

Process returned 0 (0x0)   execution time : 2.483 s
Press any key to continue.
```

Figure 1: Output of *pgm1.c* on Code::Blocks using MinGW

```
deepu@DESKTOP-J1T5JBP ~
$ gcc pgm1.c

deepu@DESKTOP-J1T5JBP ~
$ ./a.exe
02 Oct 2019 16:18:03
```

Figure 2: Output of *pgm1.c* on Cygwin using GCC

```
'pwd' is not recognized as an internal or external command,
operable program or batch file.
The current date is: 02-10-2019
Enter the new date: (dd-mm-yy)

Process returned 0 (0x0)   execution time : 1.085 s
Press any key to continue.
```

Figure 3: Output of *pgm2.c* on Code::Blocks using MinGW

```
deepu@DESKTOP-J1T5JBP ~
$ gcc pgm2.c

deepu@DESKTOP-J1T5JBP ~
$ ./a.exe
/home/deepu
02 Oct 2019 16:20:17
```

Figure 4: Output of *pgm2.c* on Cygwin using GCC

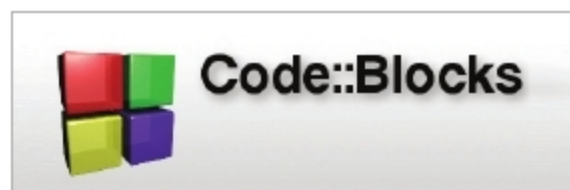


Figure 5: Logo of Code::Blocks

then prompts for a new date. Figure 2 shows the output of the program *pgm1.c* when executed with GCC on Cygwin. Note that the executable of the C program is a file named *a.exe* and not the traditional Linux executable file *a.out*. The command *cd* without a path specified doesn't do anything in Linux. Since Cygwin treats *cd* as a Linux command no action is taken in this case. The *date* command in Linux only prints the current date. On observation of the output we see that the *date* command is also treated as a Linux command and thus the current date is printed in Linux format.

Figure 3 shows the output of the program *pgm2.c* when executed with Code::Blocks and MinGW. Since *pwd* is not a valid Windows command, instead of printing the name of the current working directory, the following error message gets printed: '*pwd*' is not recognised as an internal or external command, operable program or batch file. If you look closely at the output, you will see that the *date* command is again treated as a Windows command by MinGW, and thus prints the current date in the Windows format and then prompts for a new date.

Figure 4 shows the output of the program *pgm2.c* when executed with GCC on Cygwin. Since *pwd* is a valid Linux command, the name of the current working directory is printed in this case. Please note that the current working directory that is printed starts from the root directory of Cygwin and not the absolute path printed in Figure 1 by MinGW. On observing the output, we see that the *date* command is again treated as a Linux command and thus the current date is printed in Linux format. So from the examples it is clear that MinGW and Code::Blocks know that they are working on a Windows operating system, whereas GCC and Cygwin act like they are working on the Linux operating system.

## Code::Blocks

Now that we have seen the benefits of using a MinGW compiler on Windows for programming in C, the next important question is, 'How do we set

up MinGW in Windows?'. The answer is simple. You can download and install MinGW bundled with the popular IDE Code::Blocks. A simple search on Google with the keywords Code::Blocks and MinGW will lead you to one of the many mirror sites that offer a download. The file size is typically less than 100MB, and by double clicking the executable file installation can be started in Windows. On prompting, accept MinGW as your default C compiler from the list of compilers provided by Code::Blocks. Code::Blocks is a free and open source software licensed under the GNU General Public License. It is a cross-platform IDE which can be installed on both Windows and Linux. Code::Blocks supports multiple compilers like GCC, Clang, Visual C++, etc.

Code::Blocks has a lot of built-in tools which make programming a lot easier. It has the support of the popular Linux debugger called GDB (GNU Debugger). It also has the support of Make, the Linux build automation tool. The code editor of Code::Blocks has features like syntax highlighting, code completion, etc. All these features and tools available with Code::Blocks make it a very powerful tool for C programmers. Thus the combination of MinGW and Code::Blocks is a potent weapon for practitioners of C programming in Windows. But let me add one final point to our discussion before we conclude. Even though we have learned to program with C efficiently on Windows, the best way to explore the full potential of the C programming language is by using it on the Linux operating system. **END** 🐧

By: Deepu Benson

The author is a free software enthusiast, and his area of interest is theoretical computer science. The open source tools of his choice include ns-2 and ns-3. He maintains a technical blog at [www.computingforbeginners.blogspot.in](http://www.computingforbeginners.blogspot.in).